

config_events

REFERENCE · API consumer · enable/disable the reader's event streams and configure the heartbeat

Overview

`config_events` enables or disables which device-management events the reader emits, sets the heartbeat interval and payload, and configures alert thresholds. You send a single `eventConfiguration` object of boolean flags (plus `heartbeatConfiguration` and thresholds). It is a state-changing management-plane ([MGMT](#)) command and carries no credentials.

Two things make this command distinctive:

- Changes take effect after a reboot.** The configuration is stored immediately (visible via `get_endpoint_config`), but the reader only starts emitting the configured events after a `reboot`. The command's schema states it plainly: *"To apply the changes, a device reboot is required."* Do not assume events are live until the reader has rebooted.
- Accept-but-drop.** The reader accepts the whole payload with `code 0` even if it contains flags it does not support. Of the declared flags, **7 are functional** today; the other 13 are **reserved**: accepted, stored and returned in the read-back, but with no effect on v1.1 firmware. Always read back with `get_endpoint_config` to see what was stored — but note a stored *reserved* flag has still not taken functional effect.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (updates the endpoint's stored event configuration)
Reversibility	REVERSIBLE (re-send <code>config_events</code> with the desired flags to restore a prior configuration)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>eventConfiguration</code> (with the flags)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series

Property	Value
API Version	v1.1 (API Version Reporting)
Related	<code>get_endpoint_config</code> (read back what applied), <code>config_endpoint</code> , <code>reboot</code> , <code>get_status</code>

When to Use

Use `config_events` to control the reader's management-event surface:

- **Turn event streams on/off** — enable `terminalConnection`, `firmwareUpdate`, `network`, `power`, `battery`, `fileDownload`, OR `heartbeat`.
- **Shape the heartbeat** — set `heartbeatConfiguration.interval` and choose whether each heartbeat carries inventory / battery status.
- **Reduce noise** — disable streams you do not consume so the reader emits less telemetry.

Then `reboot` to activate emission, and `get_endpoint_config` to confirm the stored configuration.

Topic Mapping

`config_events` is a **management-plane command** — answered on the MGMT/[MDM](#) endpoint, not the control endpoint.

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmd/<serial></code>	On the management endpoint that carries your session.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	Where the enabled events flow, after a reboot .

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

`config_events` configures the **event configuration of the management endpoint on which it is received** — other endpoints' event settings (set via `config_endpoint`) are left untouched.

See [Topics and Channels](#).

Key Fields

- **command** (request) — the command name; send exactly `config_events`. The schema does not `enum`-constrain it, so a wrong string is not caught by validation.
- **requestId** (request) — a unique identifier for the request; echoed back in the response to pair reply to request.
- **eventConfiguration** (request) — the event configuration to store; treat it as a **wholesale set** and send the complete configuration you want. The contract requires it.
- **enable flags** (request) — the boolean toggles inside `eventConfiguration` (`terminalConnection`, `firmwareUpdate`, `network`, `heartbeat`, `power`, `battery`, `fileDownload`, plus the reserved flags) that turn each event stream on or off; `heartbeatConfiguration` shapes the heartbeat when `heartbeat: true`.
- **response.code** (response) — `0` = Success (present on success for this management command); non-zero = error. See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `eventConfiguration`. Pick an **event set** below; the Example and Schema update together. Use the **Schema** tab for the full field reference (with the supported/reserved status per field) and the **Example** tab for the concrete payload.

Event set

Enable all events

▼

Request payload

Example: enable all supported events

```
1 {
2   "command" : "config_events" ,
3   "requestId" : "abc123" ,
4   "eventConfiguration" : {
5     "terminalConnection" : true ,
6     "firmwareUpdate" : true ,
7     "network" : true ,
8     "heartbeat" : true ,
9     "power" : true ,
10    "battery" : true ,
11    "temperature" : true ,
12    "fileDownload" : true ,
13    "heartbeatConfiguration" : {
14      "interval" : 100 ,
15      "inventoryStatus" : true ,
16      "batteryStatus" : true
17    }
18  }
19 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	comm and *	string	—	config_events	Command name. Send exactly config_events; the schema does not enum-constrain this field (see Request Semantics).
2	requ estI * d	string	—	abc123	A unique identifier for the request; echoed back in the response.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
3	eventConfiguration *	object	—	—	The event configuration to store. Required by the contract. Treat it as a wholesale set — send the complete configuration you want (see Request Semantics).
4	antenna	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
5	terminalConnection	boolean	—	true	✅ Supported. Terminal (cradle / USB-C) connect and disconnect events.
6	firmwareUpdate	boolean	—	true	✅ Supported. Firmware-update progress and completion events.
7	gpi	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
8	network	boolean	—	true	✅ Supported. Network interface state and connectivity changes.
9	exceptions	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
10	ntp	boolean	—	true	⚠ Reserved (declared for future support; not emitted on V1.1). The device may store/echo it in <u>get_endpoint_config</u> , but it has no effect yet.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
11	use rApp p	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
12	hea rtb eat	boolean	—	true	✅ Supported. Periodic heartbeat; shape it with heartbeatConfiguration.
13	pow er	boolean	—	true	✅ Supported. Power-source changes (battery / external power).
14	bat ter y	boolean	—	true	✅ Supported. Battery-state alerts (low, critical, full).
15	tem per atu re	boolean	—	true	⚠ Reserved — declared in the schema, and the reader accepts and stores it, but no temperature alert is emitted on V1.1, so switching it on has no effect (see Request Semantics).
16	fil eDo wnl oad	boolean	—	true	✅ Supported. File-download progress/completion events (firmware, certificates).
17	cpu Usa ge	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
18	fla shU sag e	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
19	ram Usa ge	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
20	heartbeatConfiguration	object	—	—	Heartbeat shape; set when heartbeat: true.
21	interval	integer	max 65535	100	✓ Supported. Seconds between heartbeats, at most 65,535 (lower = more telemetry, higher battery cost).
22	inventoryStatus	boolean	—	true	✓ Supported. Include the inventory-status block (rfidStatus, tagCount, scanCount) in each heartbeat.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
23	b a t t e r y S t a t u s	boolean	—	true	✔ Supported. Include the battery-status block in each heartbeat.
24	u s e r A p p s	boolean	—	—	⚠ Reserved — declared for future support, not functional on current V1.1 firmware.
25	cpu Thr esh old	integer	—	0	⚠ Reserved — CPU-usage percent that would trigger a CPU alert. Not honored on V1.1.
26	ram Thr esh old	integer	—	0	⚠ Reserved — RAM-usage percent. Not honored on V1.1.
27	fla shT hre sho ld	integer	—	0	⚠ Reserved — flash-usage percent. Not honored on V1.1.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
28	temperature	integer	—	0	⚠ Reserved — temperature value that would trigger the alert. Not honored on V1.1, and neither is the temperature flag itself.

When `heartbeat: true`, `heartbeatConfiguration` shapes each heartbeat message — lower `interval` means more telemetry at higher battery cost, and `inventoryStatus` / `batteryStatus` toggle which sub-blocks each heartbeat carries (see the Schema tab above for per-field detail; `userApps` is ⚠ reserved, not functional on V1.1). See [MQTT Connectivity](#) for the heartbeat event shape.

Request Semantics

Field Support. Seven of the flags below are functional on current V1.1 firmware; every flag is declared in the schema. The reader **accepts** a reserved flag (returns `code 0`) but it has no effect. The API developer has declared these for **future support**; treat them as **not currently supported**. Reserved flags are accepted, stored and echoed back in the read-back while remaining **unemitted** — **always read back with** `get_endpoint_config`, remembering that a reserved flag appearing there is stored, not functional; only the 7 supported flags take effect.

Event flag	Status	Stream
<code>terminalConnection</code>	✓ Supported	Terminal (cradle / USB-C) connect / disconnect
<code>firmwareUpdate</code>	✓ Supported	Firmware-update progress / completion
<code>network</code>	✓ Supported	Network interface / connectivity changes
<code>heartbeat</code>	✓ Supported	Periodic heartbeat (shaped by <code>heartbeatConfiguration</code>)
<code>power</code>	✓ Supported	Power-source change (battery / external)

Event flag	Status	Stream
battery	 Supported	Battery state (low, critical, full)
temperature	 Reserved	Temperature alert — declared in the schema but never sent by the reader, so switching it on is accepted and does nothing
fileDownload	 Supported	File-download progress / completion
antenna	 Reserved	Not emitted on V1.1
gpi	 Reserved	Not emitted on V1.1
exceptions	 Reserved	Not emitted on V1.1
ntp	 Reserved	Not emitted on V1.1 (may be stored/echoed)
userApp	 Reserved	Not emitted on V1.1
cpuUsage	 Reserved	Not emitted on V1.1
flashUsage	 Reserved	Not emitted on V1.1
ramUsage	 Reserved	Not emitted on V1.1

The four **threshold** fields (`cpuThreshold` , `ramThreshold` , `flashThreshold` , `temperatureThreshold`) and `heartbeatConfiguration.userApps` are likewise **reserved** — declared but not honored on V1.1. Enabling any reserved field is accepted (returns `code 0`) but has no effect. For the emitted-event catalog, see [Events](#).

- **Send every flag you want set.** Nothing in `eventConfiguration` is required and no flag has a documented default. Send the complete set of flags you intend, then check the

eventConfiguration block in `get_endpoint_config`.

Response Payload

`config_events` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. There is no configuration echo; confirm the stored configuration with `get_endpoint_config`.

Response payload

```
1 {
2   "command" : "config_events",
3   "requestId" : "abc123",
4   "apiVersion" : "V1.1",
5   "response" : {
6     "code" : 0,
7     "description" : "Success"
8   }
9 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	config_events	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — config_events echoes no configuration.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	code*	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
6	description*	string	—	Success	Human-readable status.

The contract's success example **includes** `code: 0`. `command`, `requestId`, `apiVersion` and `response` — with both `code` and `description` — are required on every reply.

Response Codes

The API contract lists **no command-specific error codes** for `config_events` — only the universal codes apply. In practice the command is highly permissive (it accepts unsupported flags with `code_0` rather than rejecting them; see [Request Semantics](#)), so success is by far the common outcome.

Code	Description
0	Success — the configuration was stored (<code>code</code> present, <code>description: "Success"</code>). Emission activates on the next reboot.
101	Default error — generic catch-all for an unhandled failure.

The contract scopes the invalid-payload code (`2`) to other commands, not to `config_events`. For the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Examples

Publish `config_events` over the MGMT endpoint; the device replies with the status envelope under [Response Payload](#), stores the supported subset immediately, and begins emitting after a reboot. The **Event set** dropdown under [Request Payload](#) shows both contract examples: **Enable**

all events, with a 100 s heartbeat, and **Selective events**, which turns some streams off

(`heartbeat: false`, no `heartbeatConfiguration`).

Notes and Caveats

- **Stored immediately, emitted after a reboot.** `config_events` writes the configuration right away — you can confirm it with `get_endpoint_config` before rebooting — but the reader only starts **emitting** the enabled events after a `reboot`. Plan a reboot into any event-configuration change.
- **Global and per-endpoint settings.** An endpoint can also carry its own `eventConfiguration`, set with `config_endpoint`. Which of the two wins when they disagree is not specified in the available sources.

`config_events` carries **no credentials** — only event flags and thresholds, so there is nothing to mask. It is **reversible**: re-send with the desired flags to restore a prior configuration (followed by a reboot). For the concept-level view, see [Event Management](#).

Related:  [get_endpoint_config](#) ·  [config_endpoint](#) ·  [reboot](#) ·  [Events](#) ·  [MQTT Connectivity](#) ·  [Event Management](#) ·  [Response Codes](#)